

第 4 次实验课作业

对 BTC 所有权的确认：钱包地址、数字签名与共识节点验签

课程	区块链原理与技术实验
学号 / 姓名	24336008 / 曾国财
完成日期	2026-09-29
实验依据	实验课 PDF 第 42-43 页

关键词：secp256k1 · Base58Check · ECDSA · 数字签名 · 所有权验证

一、实验目的与任务分解

本实验根据题目要求，模拟 Alice 向 Bob 发起转账以及共识节点确认所有权的过程。核心判断是：只有持有 Alice 私钥的一方，才能生成可被 Alice 公钥验证通过的签名。转账金额和收款地址都被编码进待签名消息，因此消息被修改后，原签名立即失效。

题目步骤	实现内容	对应代码
1. 生成两个钱包	分别创建 Alice、Bob 的 secp256k1 密钥对和地址	NewWallet()
2. 构造交易	使用 Bob 地址作为 To，金额以聪为单位保存	TransferData{To, Amount}
3. Alice 签名	对规范化交易消息做 SHA-256 后生成 DER 签名	Sign(privateKey, Message())
4. 节点验签	用 Alice 公钥验证同一消息和签名的对应关系	Verify(publicKey, Message(), sig)

表 1 实验要求与代码实现的对应关系

二、实验原理

钱包由私钥、公钥和地址组成。私钥是签名能力的根源；公钥由私钥推导得到，可公开给共识节点；地址是公钥哈希的可读编码。代码用压缩公钥执行 SHA-256 + RIPEMD-160，添加版本字节和双 SHA-256 校验和后进行 Base58 编码。

数字签名的验证逻辑

Alice 先把收款地址和金额编码成唯一消息，再用私钥签名。共识节点拿到转出公钥、原始消息和签名后，重新计算消息摘要并执行 ECDSA 验证。签名验证通过只能说明消息由对应私钥持有者签出；地址校验则只检查地址格式和校验和，不能单独证明资产所有权。

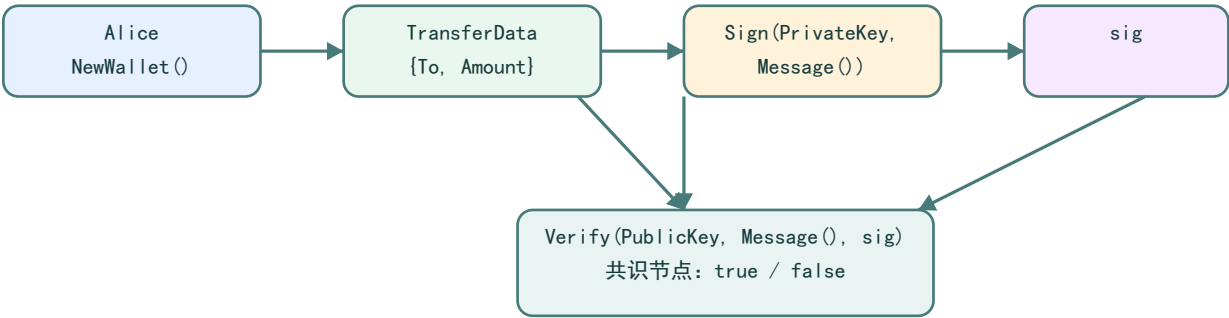


图 1 Alice 构造交易与共识节点验签流程

三、系统设计与关键实现

项目使用 Go 标准工程结构，密码学曲线使用题目示意图中的 secp256k1。第三方库只负责成熟的曲线和签名运算；地址的 Base58Check 编解码在本项目内实现，便于观察地址校验过程。

接口	作用	安全要点
NewWallet()	生成随机私钥、公钥和地址	私钥只保存在 Wallet 内，不打印、不进入交易消息
TransferData.Message()	生成带版本前缀的规范化消息	To 与 Amount 均绑定在签名摘要中
Sign()	SHA-256 + secp256k1 ECDSA，输出 DER 签名	只有私钥持有者能生成有效签名
Verify()	解析签名并用公钥验证	错误签名返回 false 或明确错误
ValidateAddress()	校验 Base58 字符、版本、长度和 checksum	防止地址输入错误，但不替代验签

表 2 核心接口设计

关键代码 (wallet.go)

```
transfer := TransferData{To: bob.Address, Amount: 50000000}
if err := transfer.Validate(); err != nil {
    log.Fatal(err)
}
sig, err := Sign(alice.PrivateKey, transfer.Message())
if err != nil {
    log.Fatal(err)
}
ok, err := Verify(alice.PublicKey, transfer.Message(), sig)
fmt.Printf("correct signature: ok=%t err=%v\n", ok, err)
```

代码清单及完整实现位于随报告提交的 wallet_validation 目录。

四、实验过程与结果

在 Go 1.26.2 环境中执行单元测试和演示程序。测试覆盖正确签名、篡改金额、错误公钥、非法 DER 签名和地址校验和变异五种情况。

4.1 单元测试输出

```
$ go test -v
=== RUN TestOwnershipVerification
--- PASS: TestOwnershipVerification (0.00s)
=== RUN TestTamperingAndWrongOwnerAreRejected
--- PASS: TestTamperingAndWrongOwnerAreRejected (0.00s)
=== RUN TestMalformedSignatureReturnsError
--- PASS: TestMalformedSignatureReturnsError (0.00s)
=== RUN TestAddressChecksumRejectsMutation
```

```
--- PASS: TestAddressChecksumRejectsMutation (0.00s)
PASS
ok      ■wallet-validation■0.168s
[exit code: 0]
```

4.2 演示程序输出

```
$ go run .
=== BTC ownership verification demo ===
Alice address: 1MtUhEqs7hZBeQDHdfJYZWevJeqpS3nsmB
Bob address:   133EkHoEAGmktidjtjb3ccG4kBzCiwe6dV
Transfer:      To=133EkHoEAGmktidjtjb3ccG4kBzCiwe6dV Amount=50000000 satoshi
Message:       BTC-TRANSFER-V1|to=133EkHoEAGmktidjtjb3ccG4kBzCiwe6dV|amount=50000000
Signature DER: 30440220733f5c53d70838b1cccf439cabd04f2de92d4d6cba0114ce9b3042204336be46022069ac26787c944e5a70f5ae7a0821faf
Address checks: Alice=true Bob=true
Correct signature: ok=true err=<nil>
Tampered amount:  ok=false err=<nil>
Wrong public key: ok=false err=<nil>
[exit code: 0]
```

结果判断：4 个单元测试全部通过；正确签名返回 `ok=true`；将金额从 50000000 修改为 50000001 后返回 `ok=false`；改用 Bob 公钥验证 Alice 的签名同样返回 `ok=false`。由此可确认签名同时绑定了签名者和交易消息。

五、结果分析与安全性讨论

验证场景	预期	实际	结论
Alice 公钥 + 原始消息 + Alice 签名	通过	ok=true	合法所有权证明
Alice 公钥 + 篡改金额 + 原签名	拒绝	ok=false	签名绑定交易内容
Bob 公钥 + 原始消息 + Alice 签名	拒绝	ok=false	签名绑定私钥持有者
地址字符或 checksum 被修改	拒绝	■■■■■	输入地址可检测

表 3 验证结果对照

本实验是交易所有权确认的最小模型，仍有三点需要在真实区块链系统中补充：第一，验证 UTXO 是否存在且未被花费；第二，检查 Alice 是否有足够余额；第三，使用交易 ID、输入引用、时间锁和区块确认数等机制防止重放与双花。本实验聚焦的是题目要求的“私钥控制权验证”，不模拟完整账本状态。

六、实验总结

本实验完成了从钱包生成、地址构造、交易消息规范化、Alice 私钥签名到共识节点公钥验签的完整闭环。实验结果说明：地址主要用于表示收款目标，真正证明转出权的是与交易消息绑定的数字签名；任何对金额、收款地址或签名者的替换都会使验证失败。实现代码可直接用 `go test -v` 和 `go run .` 复现。

七、提交文件清单

```
wallet_validation/
■■ go.mod           Go ■■■■■■■■
■■ wallet.go        ■■■■■■■■■■■■
■■ main.go          Alice -> Bob ■■■■■
■■ wallet_test.go   ■■■■■■
■■ README.md        ■■■■■■■■
■■ report/build_report.py ■■■■■■■■
```

报告依据：用户提供的《4 实验-对钱包地址的校验-20260928-2039. pdf》，重点对应第 42-43 页作业要求。